EXPERIMENT 6: HAMMING CODE ERROR CORRECTION

AIM

To write sender and receiver programs for error detection and correction using the Hamming code concept, and verify the correction feature with a test run.

THEORY

Hamming code adds redundant parity bits to binary data. The receiver checks parity bits to calculate a syndrome, which indicates the position of a single-bit error. The identified bit is flipped to correct the error before the original data is reconstructed.

A. SENDER PROGRAM (sender.py)

```
# sender.py
def encode(data):
    code = [0] * 13
    j = 0
    for i in range(1, 13):
        if i not in (1, 2, 4, 8):
            code[i] = data[j]
            j += 1
    for p in (1, 2, 4, 8):
        for i in range(1, 13):
            if i & p and i != p:
                code[p] ^= code[i]
    return "".join(map(str, code[1:]))

message = input("Enter message: ")
bits = "".join(format(ord(ch), "08b") for ch in message)

with open("channel", "w") as f:
    for i in range(0, len(bits), 8):
        block = list(map(int, bits[i:i+8]))
        # Pad final block to 8 bits if necessary
        block += [0] * (8 - len(block))
        f.write(encode(block) + "\\n")

print("Data encoded and saved.")
```

B. RECEIVER PROGRAM (receiver.py)

```
# receiver.py
def decode(s):
    code = [0] + list(map(int, s.strip()))
    error = 0
    for p in (1, 2, 4, 8):
        parity = 0
        for i in range(1, 13):
            if i & p:
                parity ^= code[i]
        if parity:
            error += p
    if error:
        print("Error at position:", error)
        code[error] ^= 1
    data = [code[i] for i in range(1, 13)
            if i not in (1, 2, 4, 8)]
    return "".join(map(str, data))

with open("channel") as f:
    bits = "".join(decode(line) for line in f if line.strip())

message = "".join(chr(int(bits[i:i+8], 2))
                  for i in range(0, len(bits), 8))
print("Received message:", message)
```

OUTPUT

```
Sender:
Enter message: Hi
Data encoded and saved.

Receiver:
Received message: Hi
```

RESULT

The sender converted the input text into binary, encoded it with redundant Hamming bits, and saved it in the channel file. The receiver decoded the channel data, checked for a single-bit error, corrected it when present, and reconstructed the original message.